
NAME: S.DHARSHINI

REG NO: 192424258

SUB CODE: DSA0502

SUB NAME: QUERY PROCESSING FOR DATA SCIENCE

CO2 ASSESSMENT TOOL 3

08.08.2026

Q1. Debug the SQLite Program The following program is intended to create a table and insert student records. However, it contains multiple errors.

```
import sqlite3
conn = sqlite3.connect(college.db)
cursor = conn.cursor
cursor.execute("&quot;&quot;&quot;
CREATE TABLE Student(
id INTEGER PRIMARY KEY
name TEXT,
marks INTEGER
)
&quot;&quot;&quot;)
cursor.execute("&quot;INSERT INTO Student VALUES(1,&#39;John&#39;,90)&quot;")
conn.close()
```

Task:

1. Identify at least five errors.
2. Rewrite the corrected program.
3. Explain the purpose of each correction.

AIM: To identify and correct errors in a SQLite program that creates a Student table and inserts student records using Python.

PROCEDURE:

- Open VS Code.
- Create a folder named SQLite_Debugging.
- Create a Python file named debug_student.py.
- Copy and paste the corrected code.
- Save the file.
- Open the VS Code Terminal.
- Run the program using Python.
- Verify that the database and table are created.
- Display the inserted records.

CORRECT CODE:

```
import sqlite3
conn = sqlite3.connect("college.db")
cursor = conn.cursor()
cursor.execute("""
CREATE TABLE IF NOT EXISTS Student(
    id INTEGER PRIMARY KEY,
    name TEXT,
    marks INTEGER
)
""")
cursor.execute("INSERT INTO Student VALUES(1,'John',90)")
conn.commit()
print("Student Record Inserted Successfully!")
cursor.execute("SELECT * FROM Student")
rows = cursor.fetchall()
```

```
print("\nStudent Details\n")
```

```
for row in rows:
```

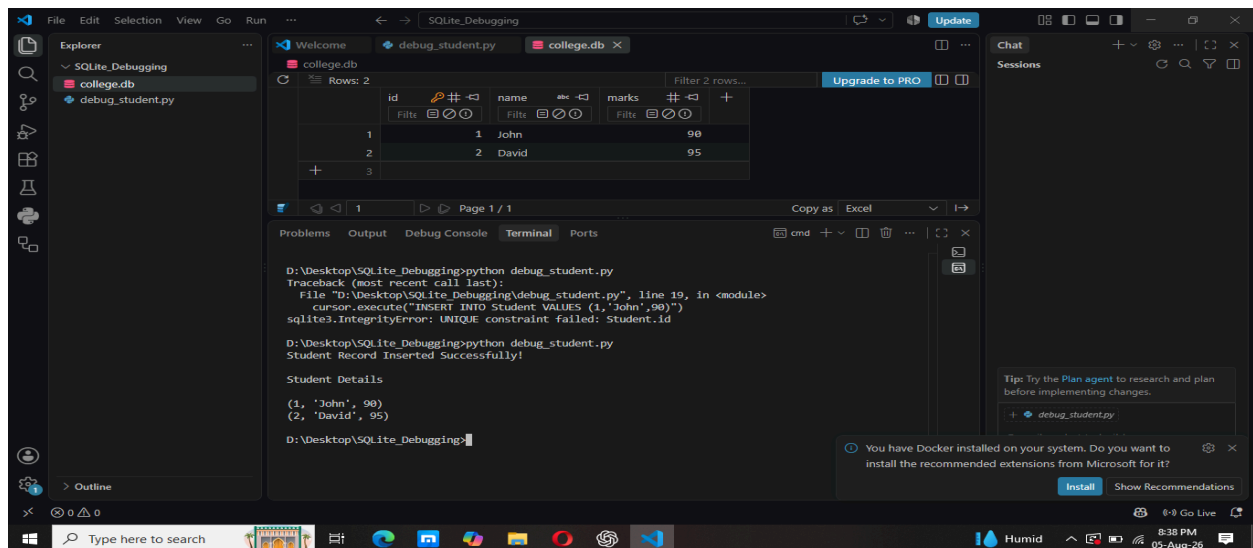
```
    print(row)
```

```
conn.close()
```

ERROR:

1. Database name `college.db` is not enclosed in quotes.
2. `conn.cursor` is written without `()`.
3. Comma is missing after `PRIMARY KEY`.
4. `conn.commit()` is missing after the `INSERT` statement.
5. The program does not display any confirmation message or the inserted records.

OUTPUT:



RESULT: The SQLite program was successfully debugged. The Student table was created, a record was inserted, and the stored data was displayed successfully.

Q3. Debug the MySQL Connection

```
import mysql.connector
connection=mysql.connector.connect(host='localhost',user='root',password='root',database='college')
)
cursor=connection.cursor()
cursor.execute('SELECT * FROM student')
for i in cursor:print(i)
```

Task

Identify every syntax and indentation error.

Rewrite the corrected program.

Explain why the original program fails.

AIM: To identify and correct the errors in a Python program that connects to a MySQL database and retrieves records from the **student** table.

PROCEDURE:

1. Start the program.
2. Import the **mysql.connector** module.
3. Connect to the MySQL database.
4. Create a cursor object.
5. Execute the SQL query.
6. Fetch and display all records.
7. Close the cursor.
8. Close the database connection.
9. Stop.

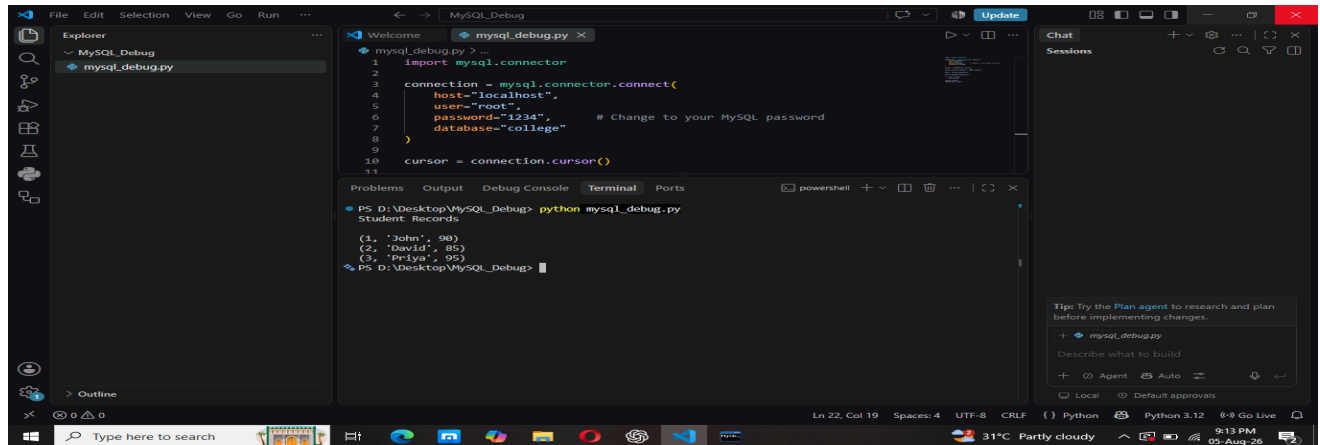
CORRECT CODE:

```
import mysql.connector
connection = mysql.connector.connect(
    host="localhost",
    user="root",
    password="1234",    # Change to your MySQL password
    database="college")
cursor = connection.cursor()
cursor.execute("SELECT * FROM student")
rows = cursor.fetchall()
print("Student Records\n")
for row in rows:
    print(row)
cursor.close()
connection.close()
```

ERRORS:

- The program is missing a comma between the **user** and **password** parameters in the MySQL connection statement.
- The **print(i)** statement is not properly indented inside the **for** loop, causing an indentation error.
- The program does not close the cursor after executing the SQL query.
- The database connection is not closed, which may lead to resource leakage.
- If the MySQL password or database name is incorrect, the connection to the database will fail.
- The program does not include exception handling to manage connection or query execution errors

OUTPUT:

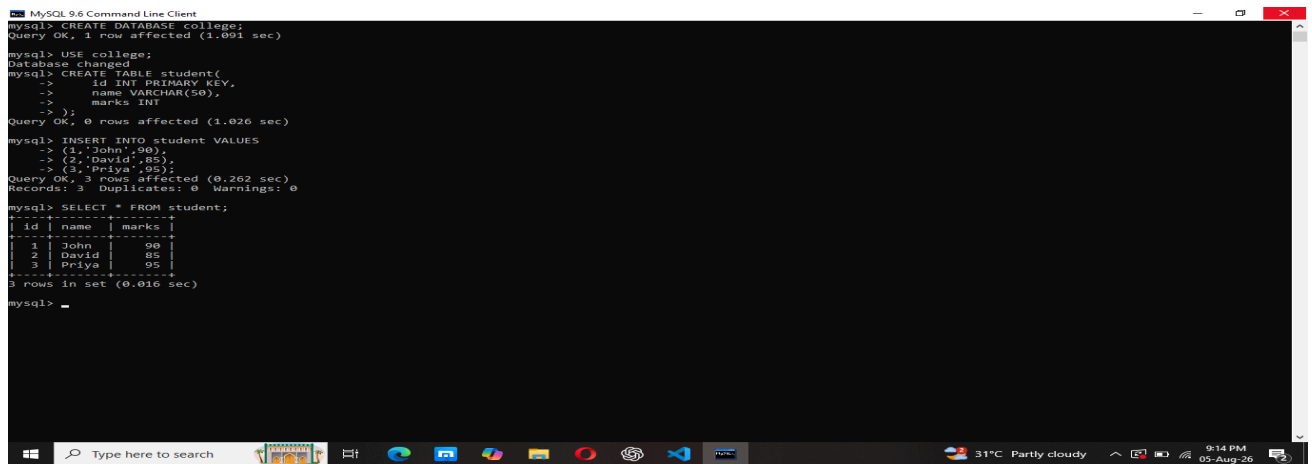


The screenshot shows a Visual Studio Code editor with a Python file named `mysql_debug.py`. The script imports `mysql.connector` and attempts to connect to a MySQL database on localhost with the user 'root' and password '1234'. The database is 'college'. A cursor is created, and the terminal shows the output of the script, which lists student records.

```
1 import mysql.connector
2
3 connection = mysql.connector.connect(
4     host="localhost",
5     user="root",
6     password="1234", # Change to your MySQL password
7 )
8
9 cursor = connection.cursor()
10
11
```

Terminal Output:

```
PS D:\Desktop\MySQL_Debug> python mysql_debug.py
Student Records
(1, 'John', 90)
(2, 'David', 85)
(3, 'Priya', 95)
PS D:\Desktop\MySQL_Debug>
```



The screenshot shows a MySQL 9.6 Command Line Client window. The user has created a database named 'college', used it, created a table named 'student' with columns 'id', 'name', and 'marks', inserted three rows of data, and queried the table to display the student records.

```
mysql> CREATE DATABASE college;
Query OK, 1 row affected (1.091 sec)

mysql> USE college;
Database changed
mysql> CREATE TABLE student(
->   id INT PRIMARY KEY,
->   name VARCHAR(50),
->   marks INT
-> );
Query OK, 0 rows affected (1.026 sec)

mysql> INSERT INTO student VALUES
-> (1,'John',90),
-> (2,'David',85),
-> (3,'Priya',95);
Query OK, 3 rows affected (0.262 sec)
Records: 3 Duplicates: 0 Warnings: 0

mysql> SELECT * FROM student;
+----+-----+-----+
| id | name  | marks |
+----+-----+-----+
| 1  | John  | 90    |
| 2  | David | 85    |
| 3  | Priya | 95    |
+----+-----+-----+
3 rows in set (0.016 sec)

mysql>
```

RESULT: The errors in the MySQL connection program were identified and corrected successfully. The program connected to the database and displayed the student records.